



《AI大模型Ragent项目》——分布式限流选业务层还是网关层？

来自： 拿个offer-开源&项目实战

 马丁

2026年03月29日 12:41

前面两篇文章解决了文件上传的两个关键问题：通过 Spring Boot 配置限制单文件大小（max-file-size: 50MB）和请求大小（max-request-size: 100MB），避免超大文件攻击；通过预签名 URL + 流式上传优化内存占用，30MB 文件不会占用 100MB 堆内存。上一篇文章又解决了并发控制问题，用 RPermitExpirableSemaphore 限制同时上传的文件数，防止磁盘空间膨胀和 IO 飙升。

但还有一个关键问题没解决：**限流代码应该放在哪一层？**

假设你配置了单文件最大 50MB，Tomcat 最大线程数 200。如果某个时刻出现了集中上传、批量导入，或者客户端重试放大导致 200 个请求同时进来，即使你在 Service 层做了限流，是不是意味着 $200 \times 50MB = 10GB$ 的临时文件已经产生了？磁盘和 IO 还是会瞬间承压。这个问题的答案取决于限流代码放在哪个环节。如果放在 Controller 或 Service 层，确实太晚了，文件已经解析完成，临时文件已经写入磁盘。但如果放在 Filter 层或 Gateway 层，情况就完全不同。

这篇文章讲清楚限流位置的选择：从 Tomcat 请求处理流程出发，逐层分析 Service、Filter、Gateway 三个位置的限流效果，用极端场景验证各层方案的实际表现，最后给出单体应用和微服务架构的选型建议。

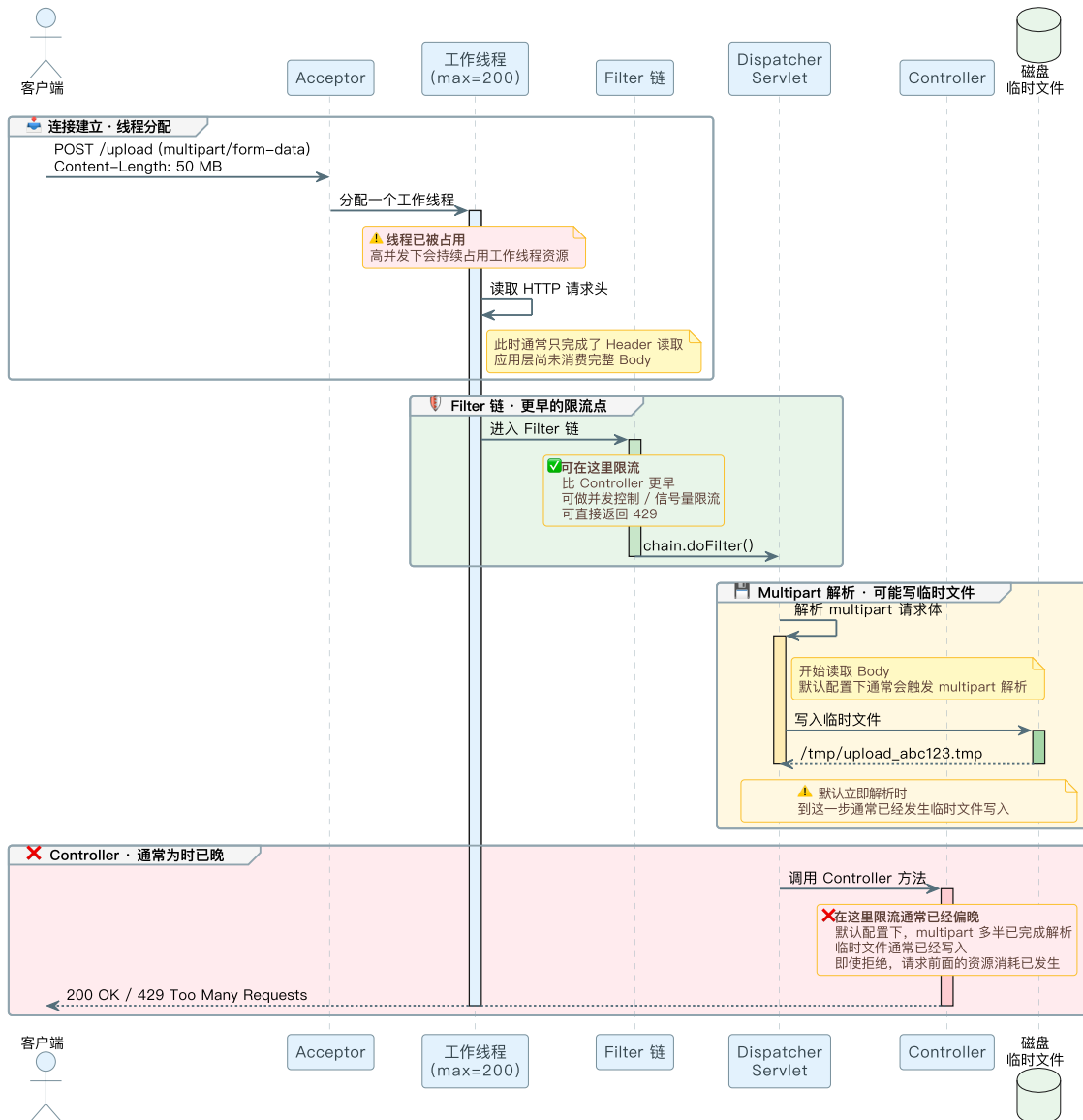
Tomcat 请求处理流程

要理解限流应该放在哪，先要搞清楚 Tomcat 处理文件上传请求的完整流程。很多人对这个流程有误解，以为 Tomcat 收到请求就立刻解析 multipart，把文件写到磁盘。实际上不是这样的。

1. 完整请求链路

从客户端发起上传请求，到 Controller 方法被调用，中间经过了很多环节。用时序图展示完整链路：

Tomcat 文件上传：一个请求到底经历了什么？



这个流程图标注了三个关键时机：

1. **线程分配**：发生在 Filter 之前，即使在 Filter 里快速返回 429，线程还是被占用了
2. **Filter 层**：在 multipart 解析之前，可以拦截请求，防止文件被写入磁盘
3. **Controller 层**：multipart 已经解析完成，临时文件已经产生，限流太晚了

2. multipart 解析时机

很多人对 multipart 解析时机有误解，以为 Tomcat 收到请求就立刻把整个文件读到内存或磁盘。实际上，Tomcat 的请求处理是分阶段的。

2.1 请求体的懒读取

HTTP 请求分为请求头和请求体两部分。Tomcat 在接收到连接后，会先读取请求头，此时请求体（也就是文件数据）还在 socket buffer 里，没有被完整读取。

这个机制叫懒读取（Lazy Loading）。只有当应用代码调用 `request.getParts()` 或 `request.getInputStream()` 时，Tomcat 才会开始读取请求体。

对于 multipart 请求，Spring Boot 的 `DispatcherServlet` 会在内部调用 `request.getParts()` 来解析文件，这个时候才会触发文件读取和临时文件写入。

2.2 DispatcherServlet 的作用

`DispatcherServlet` 是 Spring MVC 的核心组件，负责请求分发。它在 Filter 链执行完成后才会被调用。

`DispatcherServlet` 内部会检查请求的 `Content-Type`，如果是 `multipart/form-data`，就会调用 `MultipartResolver` 来解析文件。默认情况下，Spring Boot 使用 `StandardServletMultipartResolver`，它会调用 Servlet 3.0 的 `request.getParts()` 方法。

`request.getParts()` 方法会做两件事：

1. 从 socket 读取请求体数据

- 2. 把文件数据写到临时目录（java.io.tmpdir）

所以，multipart 解析发生在 DispatcherServlet 内部，Filter 之后，Controller 之前。

2.3 常见误区

误区一：Tomcat 收到请求就立刻解析 multipart

不对。Tomcat 只是建立了连接，读取了请求头，请求体还在 socket buffer 里。只有 DispatcherServlet 调用 request.getParts() 时，才会触发解析。

误区二：在 Filter 里返回 429，文件还是会被读取

不对。如果在 Filter 里直接返回 429，不调用 chain.doFilter()，请求就不会到达 DispatcherServlet，multipart 不会被解析，文件不会被写入磁盘。

误区三：HandlerInterceptor 可以在 multipart 解析之前拦截

不对。HandlerInterceptor 是在 DispatcherServlet 内部执行的，此时 multipart 已经解析完成，临时文件已经产生。用 HandlerInterceptor 做限流和在 Controller 里做限流本质上是一样的，都太晚了。

3. 线程分配的时机

线程分配发生在 Tomcat Acceptor 接收到连接之后，Filter 执行之前。

Tomcat 的线程模型是这样的：

- 1. **Acceptor 线程**：负责接收新连接，数量很少（通常 1-2 个）
- 2. **工作线程池**：负责处理请求，默认最大 200 个线程

当客户端发起连接时，Acceptor 线程会接收这个连接，然后从工作线程池中分配一个线程来处理这个请求。这个线程会一直占用，直到请求处理完成并返回响应。

所以，即使你在 Filter 里快速返回 429，这个线程还是被占用了。200 个并发请求会占满 200 个工作线程。

这就是为什么 Filter 层限流虽然能防止临时文件产生，但无法防止线程被占用。要真正防止线程被占用，需要在请求到达 Tomcat 之前就拦截，也就是 Gateway 层。

各层限流方案对比

理解了 Tomcat 的请求处理流程，现在逐层分析限流方案。从最晚的 Service 层开始，一直到 Gateway 层，看看每一层的限流效果如何。

1. Service 层限流

1.1 实现方式

在 Service 方法内部，文件上传到对象存储之前获取信号量。严格来说，Controller 层和 Service 层在这个问题是同一类方案，都是进入业务代码之后才开始限流，所以这里合并讨论，只保留 Service 层。

1.2 问题分析

当代码执行到 Service 方法时，文件已经被 DispatcherServlet 解析完成，临时文件已经产生。信号量只能控制后续的对象存储上传，但磁盘和 IO 的问题已经发生了。

极端场景下的表现：

- 200 个并发请求全部到达 Tomcat
- 200 个工作线程被占用
- 10GB 临时文件产生
- 磁盘 IO 被打满

结论：Service 层限流无法解决问题，不推荐。

2. Filter 层限流

2.1 实现方式

在 Filter 中，DispatcherServlet 之前拦截。这是单体应用中推荐的做法。

Filter 在 Servlet 规范中的执行顺序是：Filter 链 → Servlet（DispatcherServlet）→ Controller。如果在 Filter 中拦截请求，不调用 chain.doFilter()，请求就不会到达 DispatcherServlet，multipart 不会被解析。

2.2 代码示例

完整的 UploadRateLimitFilter 实现：

```
/**
 * 文件上传限流 Filter
 * 在 multipart 解析之前拦截，防止临时文件产生
```

```

*/
@Slf4j
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
@RequiredArgsConstructor
public class UploadRateLimitFilter extends OncePerRequestFilter {

    private final RedissonClient redissonClient;
    private final RagSemaphoreProperties semaphoreProperties;

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request,
                                    @NonNull HttpServletResponse response,
                                    @NonNull FilterChain chain) throws ServletException, IOException {

        // 判断是否是上传请求
        if (!isUploadRequest(request)) {
            chain.doFilter(request, response);
            return;
        }

        // 获取信号量配置
        RagSemaphoreProperties.PermitExpirableConfig config = semaphoreProperties.getDocumentUploadPermitExpirableSemaphore(semaphore = redissonClient.getPermitExpirableSemaphore(config.ge

        // 尝试获取许可
        String permitId = null;
        try {
            permitId = semaphore.tryAcquire(
                config.getMaxWaitSeconds(),
                config.getLeaseSeconds(),
                TimeUnit.SECONDS
            );

            if (permitId == null) {
                // 获取许可失败，直接返回 429
                response.setStatus(429);
                response.setContentType("application/json;charset=UTF-8");
                response.getWriter().write("{\"code\":\"429\",\"message\":\"当前上传人数过多，请稍后再试\"}");
                return; // 不调用 chain.doFilter(), 请求到此为止
            }

            // 获取许可成功，继续处理请求
            chain.doFilter(request, response);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            response.setStatus(500);
            response.setContentType("application/json;charset=UTF-8");
            response.getWriter().write("{\"code\":\"500\",\"message\":\"获取上传许可失败\"}");
        } finally {
            if (permitId != null) {
                boolean released = semaphore.tryRelease(permitId);
                if (!released) {
                    log.warn("upload permit already expired or released, permitId={}", permitId);
                }
            }
        }
    }

    private static final String UPLOAD_PATH_PATTERN = "/knowledge-base/";
    private static final String UPLOAD_PATH_SUFFIX = "/docs/upload";

    /**
     * 判断是否是文档上传请求
     */
}

```

```
private boolean isUploadRequest(HttpServletRequest request) {
    if (!"POST".equals(request.getMethod())) {
        return false;
    }
    String uri = request.getRequestURI();
    return uri != null && uri.contains(UPLOAD_PATH_PATTERN) && uri.endsWith(UPLOAD_PATH_SUFFIX)
}
}
```

代码关键点说明：

1. **继承** OncePerRequestFilter：确保每个请求只执行一次，避免重复拦截
2. @Order(Ordered.HIGHEST_PRECEDENCE)：设置最高优先级，确保在其他 Filter 之前执行
3. isUploadRequest() **判断**：只拦截 POST 请求且等于知识库文档上传 URI 的请求
4. **不调用** chain.doFilter()：如果获取许可失败，直接返回 429，不继续执行后续的 Filter 和 Servlet
5. **try-finally 结构**：确保许可一定会被释放，即使处理过程中抛出异常

2.3 优势分析

Filter 层限流的核心优势是：**在 multipart 解析之前拦截，不会写临时文件。**

当 Filter 返回 429 时，请求不会到达 DispatcherServlet，request.getParts() 不会被调用，文件数据不会从 socket 读取，临时文件不会被写入磁盘。

极端场景下的表现：

- 200 个并发请求全部到达 Tomcat
- 200 个工作线程被占用（这个无法避免）
- 但只有 10 个请求（假设信号量许可数为 10）会触发 multipart 解析
- $10 \times 50\text{MB} = 500\text{MB}$ **临时文件**，可控
- 其余 190 个请求在 Filter 中被拒绝，直接返回 429，不会触发文件解析

Filter 层限流能有效防止磁盘空间膨胀和 IO 飙升，这是它最大的价值。

2.4 局限性

Filter 层限流虽然能防止临时文件产生，但有一个无法解决的问题：**线程还是被占用了。**

线程分配发生在 Filter 执行之前。当 200 个并发请求到达 Tomcat 时，Tomcat 会从工作线程池中分配 200 个线程来处理这些请求。即使 Filter 快速返回 429，这 200 个线程还是被占用了。

如果 Tomcat 的最大线程数是 200，这 200 个线程全部被上传请求占用，其他正常请求就无法处理了。虽然上传请求很快就返回了（Filter 里直接返回 429，不需要等待文件上传），但在高并发场景下，还是会影响系统的吞吐量。

虽然影响系统吞吐量的范围很小，但还是会有影响。有影响的话，自然要看怎么更优雅的解决问题。

结论：Filter 层限流是单体应用的最佳选择，能有效防止磁盘和 IO 问题，但无法防止线程被占用。

3. Gateway 层限流

3.1 Gateway 的流式透传原理

Spring Cloud Gateway 是微服务架构中常用的网关组件。它和 Tomcat 有本质区别：Tomcat 是基于 Servlet 的阻塞式 IO，而 Gateway 是基于 Netty + WebFlux 的响应式非阻塞 IO。

这个区别导致 Gateway 处理文件上传的方式完全不同。

Tomcat 的处理方式：

1. 接收到请求，分配一个线程
2. 这个线程阻塞等待，从 socket 读取完整的请求体
3. 把文件数据写到临时文件
4. 调用 Controller 处理
5. 返回响应，释放线程

Gateway 的处理方式：

1. 接收到请求，不会像 Tomcat 那样为每个请求独占一个工作线程
2. 从 socket 读取一小块数据（比如 8KB），封装成 DataBuffer

3. 把这个 `DataBuffer` 转发给后端服务
4. 继续读取下一块数据，继续转发
5. 循环往复，直到整个文件传输完成

Gateway 不会像 Servlet 容器那样把整个文件先解析成 multipart，也不会先落到临时文件。它更像一个搬运工，把数据从客户端搬到后端服务，一块一块地搬。

用代码表示，Gateway 处理的请求体是 `Flux<DataBuffer>`：

```
// Tomcat 的方式：整个文件作为一个对象
MultipartFile file = request.getFile("file");
byte[] bytes = file.getBytes(); // 整个文件在内存里

// Gateway 的方式：文件是一个数据流
Flux<DataBuffer> body = request.getBody();
body.subscribe(dataBuffer -> {
    // 每次收到一小块数据（比如 8KB）
    // 立即转发给后端，不会攒成一个大文件
});
```

这就是流式透传。Gateway 不关心传输的是什么内容，不会解析 multipart，只是把字节流从客户端搬到后端。

3.2 工作流程

用一个具体的例子说明 Gateway 如何处理 50MB 文件上传：

1. **客户端开始上传**：发送 POST 请求，Content-Length: 50MB
2. **Gateway 收到请求头**：先处理 HTTP 请求头，拿到路由、路径，以及可能存在的 Content-Length
3. **Gateway 开始转发**：
 - 从客户端 socket 读取 8KB 数据 → 封装成 DataBuffer
 - 把这个 DataBuffer 写到后端服务的 socket
 - 从客户端 socket 读取下一个 8KB 数据 → 封装成 DataBuffer
 - 把这个 DataBuffer 写到后端服务的 socket
 - 循环往复，直到 50MB 全部传输完成
4. **后端服务收到数据**：Tomcat 从 socket 读取数据，解析 multipart，写临时文件
5. **后端服务返回响应**：Gateway 收到响应，转发给客户端

在这个过程中，Gateway 自身通常只会持有少量 DataBuffer，比如 3–5 个 8KB 的 buffer，总共几十 KB。它不会像 Servlet 容器那样把 50MB 文件先落到本地临时文件。

3.3 实现方式

如果你的目标是**控制同时上传的文件数**，不要直接把 Spring Cloud Gateway 的 `RequestRateLimiter` 当成并发控制来用。

`RequestRateLimiter` 控制的是单位时间内的请求速率，本质上是限速，不是“当前正在上传的请求数”。

真正要做并发控制，更合适的做法是自定义 `GatewayFilter`，在网关里复用和 Filter 层同样的分布式信号量逻辑：

1. 识别上传请求
2. 在转发前尝试获取分布式信号量许可
3. 获取失败直接返回 429，不再继续路由和转发
4. 获取成功后继续转发，并在请求处理结束后释放许可

如果你还想额外控制突发请求速率，可以再叠加 `RequestRateLimiter`。但那是“限速”，不是本文讨论的“并发上传数控制”。

3.4 优势分析

Gateway 层限流的核心优势是：**如果限流判断只依赖路由、路径、用户标识等请求头阶段就能拿到的信息，就可以在不进入后端业务服务的情况下直接拒绝请求。**

当 Gateway 拒绝一个上传请求时：

1. Gateway 先处理 HTTP 请求头和路由信息
2. Gateway 执行限流逻辑，发现超过并发限制
3. Gateway 直接返回 429 响应
4. Gateway 关闭与客户端的连接

5. 请求不会再被转发到后端服务，也不会进入后端服务的 multipart 解析流程

这和 Filter 层限流不同。Filter 层限流时，虽然 multipart 没有被解析，但请求已经到达了 Tomcat，线程已经被占用了。而 Gateway 层限流时，请求根本没有到达后端服务，后端服务的 Tomcat 线程完全不受影响。

极端场景下的表现：

200 个并发请求到达 Gateway

假设 Gateway 用分布式信号量把并发上传数限制为 10

最终只有 10 个请求到达后端服务

后端服务的 Tomcat 只有 10 个线程被占用

后端服务只产生 10 × 50MB = **500MB 临时文件**

其余 190 个请求在 Gateway 被拒绝，直接返回 429

Gateway 自身只占用少量 buffer 和 Netty 线程。被拒绝的请求不会占用后端服务的线程，也不会进入后端服务的临时文件生成流程

这里有个前提要说清楚：上面的结论成立，是因为我们假设 Gateway 只是做路由和限流，没有额外启用会缓存或改写请求体的过滤器。如果网关侧主动读取、缓存 body，资源占用模型就会变。

对比 Filter 层限流：

对比项	Filter 层限流	Gateway 层限流
临时文件规模	最多 10 × 50MB = 500MB	最多 10 × 50MB = 500MB
磁盘 IO	10 个文件同时写入	10 个文件同时写入
后端线程占用	200 个线程	10 个线程
网关自身资源占用	-	少量 DataBuffer 和事件循环线程

Gateway 层限流不仅能防止磁盘和 IO 问题，还能防止后端服务的线程被占用。这是它最大的优势。

各层方案对比表格

把三种限流方案放在一起对比，看看各自的优缺点：

限流位置	拦截时机	临时文件规模	后端线程占用	内存占用	实现复杂度	适用场景	推荐指数
Service 层	Service 方法内	❌ 不可控 (10GB)	❌ 已占用 (200 线程)	❌ 已占用	★ 简单	❌ 不推荐	☆☆☆☆☆
Filter 层	multipart 解析前	✅ 可控 (最多 500MB)	❌ 已占用 (200 线程)	✅ 较低	★★ 中等	✅ 单体应用	★★★★☆
Gateway 层	到达后端服务前	✅ 可控 (最多 500MB)	✅ 可控 (10 线程)	✅ 网关仅少量 buffer	★★★★ 复杂	✅ 微服务架构	★★★★★

选型建议

根据不同的架构，给出限流位置的选型建议。

1. 单体应用架构

如果你的系统是单体应用，没有使用微服务架构，推荐把限流放在 **Filter 层**。对于内部知识库，这通常已经是性价比最高、实现也最稳妥的做法。

推荐方案：Filter 层限流

在应用内部使用 Filter + 分布式信号量，控制集群级别的并发数：

```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class UploadRateLimitFilter extends OncePerRequestFilter {
    // 使用 RPermitExpirableSemaphore 控制全局并发数
```

```
// 比如整个集群最多 10 个并发上传
}
```

这一层主要控制应用内部的总并发数。比如某个时刻有很多同事同时上传文件，或者某个批量导入任务叠加了人工操作，总并发数仍然可能偏高。Filter 层的信号量可以在集群级别控制总并发数。

为什么单体应用推荐 Filter 层：

- 能在 multipart 解析之前拦截，直接解决临时文件和磁盘 IO 问题
- 实现位置离业务最近，配置和观测都比较直接
- 对内部系统来说，通常没有必要再为了这件事额外引入更前面的限流层

配置示例：

```
rag:
  semaphore:
    document-upload:
      max-concurrent: 10          # 全局最多 10 个并发
      max-wait-seconds: 5
      lease-seconds: 300
```

这样配置后，极端场景下的表现：

- 整个集群最多 10 个并发上传
- 临时文件最多 10 × 50MB = 500MB
- 磁盘 IO 可控
- 后端线程占用可控（虽然会被占用，但很快释放）

2. 微服务架构

如果你的系统使用了微服务架构，有 Spring Cloud Gateway 或其他网关，推荐把限流放在 Gateway 层。

推荐方案：Gateway 层限流

在 Gateway 自定义并发控制过滤器，控制集群级别的并发数：

做法和 Filter 层类似，区别只是执行位置前移到了 Gateway：在请求真正路由到业务服务之前，先获取分布式信号量；获取失败直接返回 429；获取成功再继续转发，并在请求结束后释放许可。

补充方案：Filter 层限流（可选）

如果你希望把保护做得更细，也可以在业务服务内部再加一层 Filter 限流。但通常不一定需要，因为 Gateway 层已经拦截了大部分请求。

为什么微服务架构推荐 Gateway 层：

- Gateway 是微服务架构的标配，所有请求都会经过 Gateway
- Gateway 层限流可以在请求到达业务服务之前拦截，不占用业务服务的线程
- Gateway 流式透传，网关自身通常只占用少量 buffer，不会像后端 Servlet 容器那样先落本地临时文件
- 如果用 Filter 层限流，虽然能防止临时文件，但业务服务的线程还是会被占用

极端场景下的表现：

- 整个集群最多 10 个并发上传（Gateway 层）
- 只有 10 个请求到达业务服务
- 业务服务只有 10 个线程被占用
- 临时文件最多 10 × 50MB = 500MB
- Gateway 自身通常只占用少量 buffer，不会像后端 Servlet 容器那样先落本地临时文件

3. 为什么 Filter 和 Gateway 不是互斥关系

有人可能会问：既然微服务里 Gateway 层限流已经足够靠前，为什么有时还会提到 Filter 层限流？

原因一：控制位置不同

- Gateway 层限流：在请求进入业务服务之前拦截，优先保护后端线程
- Filter 层限流：在业务服务内部兜底，优先保护 multipart 解析和临时文件写入

如果 Gateway 已经能稳定拦住大部分上传请求，通常不需要再加 Filter。但如果上传链路里还有绕过 Gateway 的入口，或者你希望业务服务内部也有一层兜底，Filter 仍然有价值。

原因二：限流粒度不同

- Gateway 层限流：更适合做入口级、服务级的总量控制
- Filter 层限流：可以按用户、按知识库、按文件类型限制，粒度更细

比如，你可以在 Filter 层实现：

- VIP 用户不限流，普通用户限流
- 单个知识库最多 5 个并发上传
- PDF 文件限流，TXT 文件不限流

这些细粒度的限流逻辑，通常还是要在应用层实现。

原因三：动态调整

- Gateway/Filter 层限流：都可以从配置中心动态读取，实时调整

比如，你发现系统负载很高，想临时把并发数从 10 降到 5。用 Gateway 或 Filter 限流时，都可以通过配置中心或动态配置快速生效。

结论：Filter 和 Gateway 不是谁一定替代谁，而是看架构位置。单体应用优先用 Filter，微服务优先用 Gateway；只有在确实需要更细粒度业务兜底时，才考虑两层一起用。

小结

这篇文章从 Tomcat 请求处理流程出发，逐层分析了 Service、Filter、Gateway 三个位置的限流方案。

核心要点：

- 限流越早越好，但要考虑实现复杂度：**在这篇文章讨论的几层里，Gateway 是更靠前的拦截点，Filter 则更适合作为单体应用里的关键兜底点。
- Service 层限流完全不推荐：**Controller 层和它本质一样，都是进入业务代码之后才开始限流。这一类方案都会在临时文件已经产生之后才生效，无法解决磁盘和 IO 问题。
- Filter 层限流适合单体应用：**能在 multipart 解析之前拦截，防止临时文件产生，但线程还是会被占用。极端场景下，临时文件从 10GB 降到 500MB，磁盘 IO 可控。
- Gateway 层限流适合微服务架构：**能在请求到达业务服务之前拦截，不仅防止临时文件，还能防止线程被占用。Gateway 自身只需要维持少量 buffer 和事件循环线程。
- 选型建议很直接：**单体应用优先用 Filter，微服务架构优先用 Gateway；如果微服务里还需要业务服务内部兜底，再补一层 Filter。

如果后面真的要追求极致优化，也可以再把这件事前移到 Nginx 层。这样连 Gateway 都不需要做无意义的转发和执行了。不过对于当前这个内部知识库场景，这不是正文里的默认推荐方案。